Lớp: IT003.Q21.CTTN

BÁO CÁO KẾT QUẢ THỬ NGHIỆM

Sinh viên thực hiện: Lê Phú Trọng (25521937)

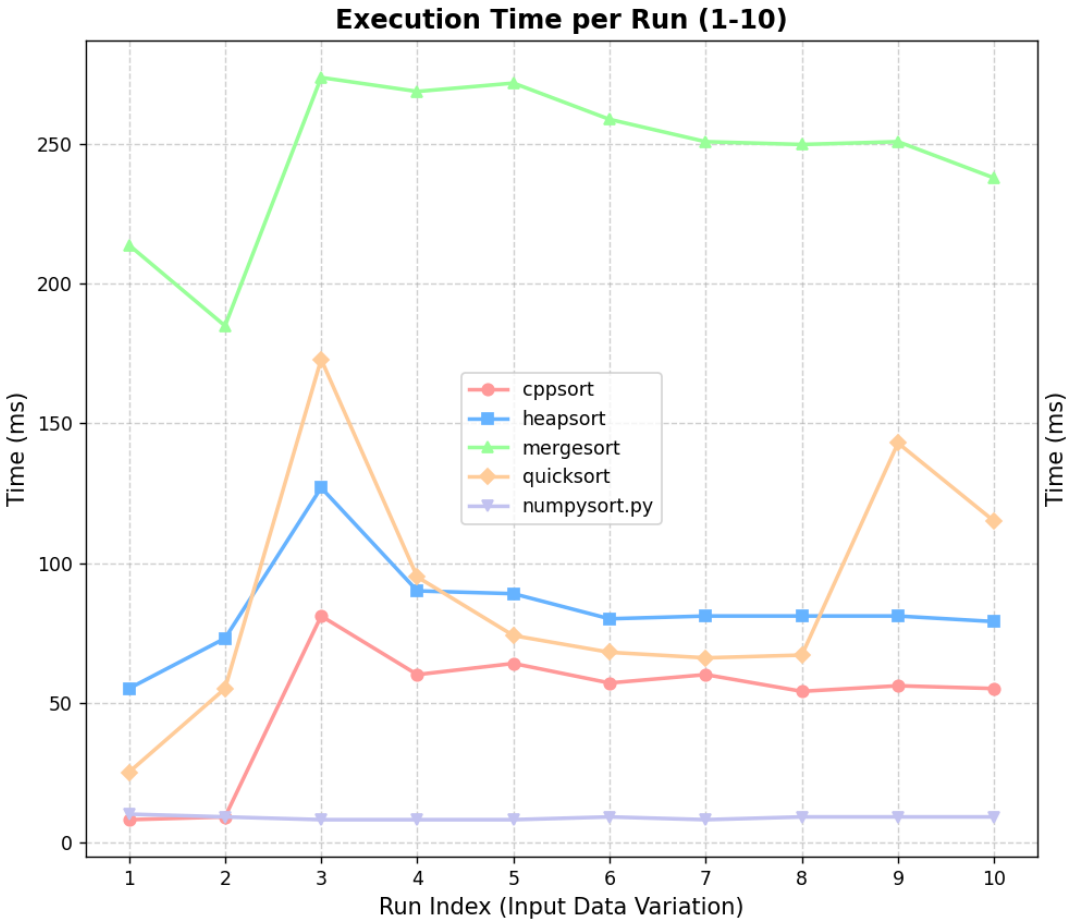
Nội dung báo cáo: Thực nghiệm các giải thuật sắp xếp nội

I. Kết quả thử nghiệm

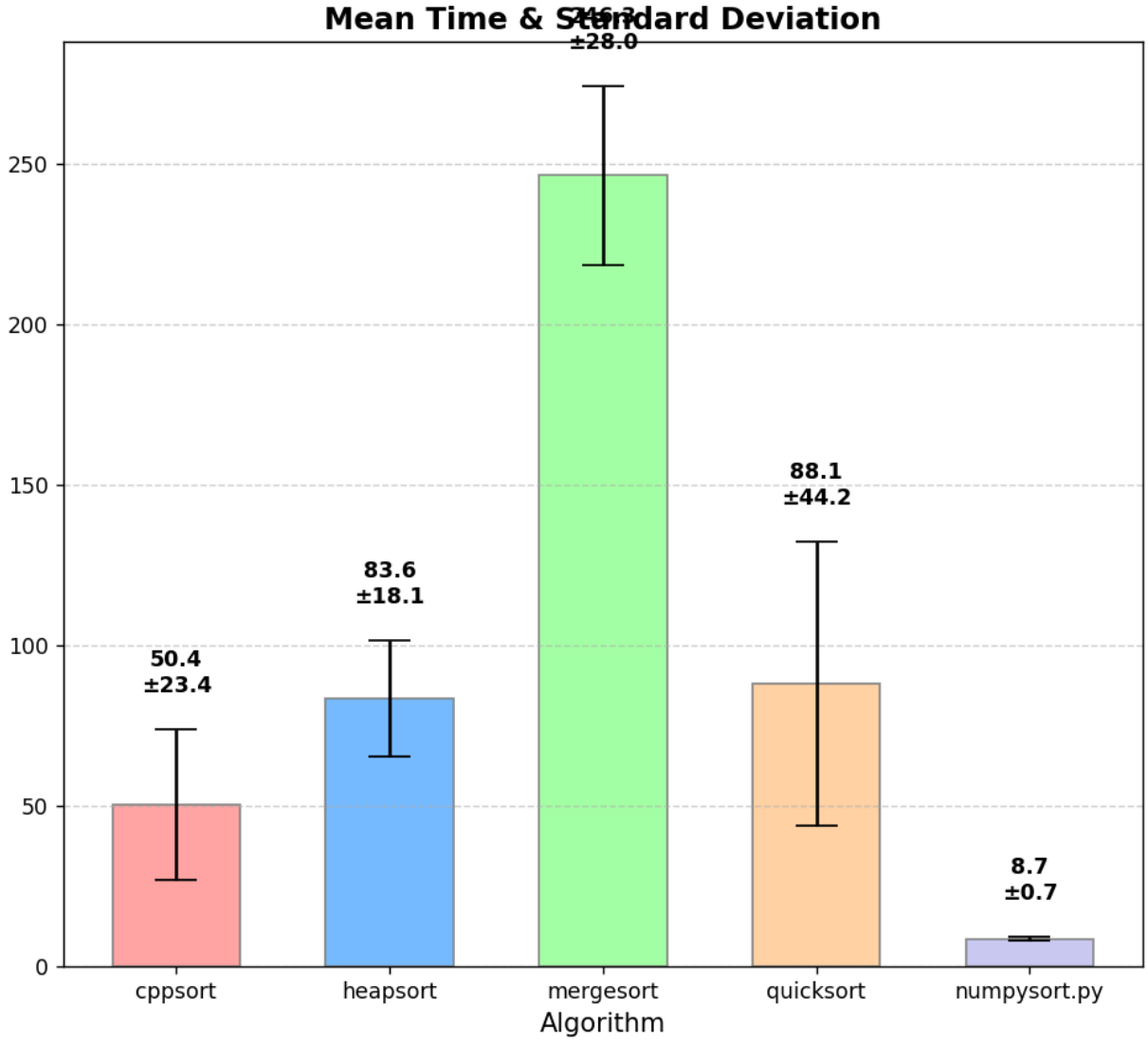
1. Bảng thời gian thực hiện

Dữ liệu	**Thời gian thực hiện (ms)**				
	Quicksort	**Heapsort**	**Mergesort**	**sort (C++)**	**sort (numpy)**
1	25	55	214	8	10
2	55	73	185	9	9
3	173	127	274	81	8
4	95	90	269	60	8
5	74	89	272	64	8
6	68	80	259	57	9
7	66	81	251	60	8
8	67	81	250	54	9
9	143	81	251	56	9
10	115	79	238	55	9
Trung bình	88.1	83.6	246.3	50.4	8.7
Độ lệch chuẩn	44.2	18.1	28	23.4	0.7

2. Biểu đồ thời gian thực hiện



Hình 1: Biểu đồ đường thẳng thời gian chạy các thuật toán qua 10 tập dữ liệu



Hình 2: Biểu đồ cột thời gian trung bình và độ lệch chuẩn của từng loại thuật toán

II. *Kết luận*

Dựa trên thời gian chạy trung bình, thứ hạng các thuật toán lần lượt là: Numpy sort, C++ sort, Heapsort, Quicksort (random pivot), Mergesort.

Sự chênh lệch lớn giữa numpy sort, C++ sort cho thấy dù cùng có độ phức tạp là $O(N \log N)$, thời gian chạy vẫn có sự khác biệt do các thuật toán của thư viện đã được tối ưu ở các tầng thấp, cụ thể:

- Numpy không chạy Python mà gọi xuống thư viện C đã được tối ưu hóa vector hóa (SIMD instruction), tận dụng tối đa Cache của CPU [1].
- Trong khi đó, C++ sort là thuật toán lai giữa Quicksort, Heapsort và Insertion Sort [2].

Tác động của dữ liệu cũng ảnh hưởng đến kết quả:

- C++ sort hoạt động nhanh hơn khoảng gấp 3 lần bình thường nhờ vào cơ chế Branch Prediction của CPU, hoạt động hiệu quả đối với dữ liệu có quy luật.
- Ngược lại, Quicksort chịu ảnh hưởng tiêu cực nặng nhất, với dữ liệu giảm dần, chi phí quá lớn khi phải đảo ngược toàn bộ mảng cho dù đã sử dụng Random

Pivot. Nếu không sử dụng Random Pivot mà sử dụng phần tử cuối, độ phức tạp thực tế đạt đến $O(N^2)$.

Độ ổn định:

- Mergesort: Mặc dù chậm nhất nhưng độ lệch chuẩn thấp, cho thấy đây là một thuật toán khá an toàn, không phụ thuộc nhiều vào dữ liệu đầu vào. Ngoài ra đây cũng là thuật toán có ý tưởng dễ hiểu và dễ cài đặt nhất trong các thuật toán.
- Heapsort: Mang lại hiệu quả tốt, với độ lệch chuẩn khá thấp, không phụ thuộc nhiều vào dữ liệu đầu vào. Tuy nhiên thuật toán phải sử dụng đến cấu trúc dữ liệu Heap để cài đặt.
- Quicksort: Thuật toán bất ổn định nhất, dựa hoàn toàn vào việc chọn Pivot. Điều này khẳng định Quicksort thuần túy vẫn tiềm ẩn rủi ro về hiệu năng trong các hệ thống thực tế.

III. *Thông tin chi tiết*

Thông tin chi tiết hơn về báo cáo, mã nguồn và dữ liệu thử nghiệm có thể được xem tại: https://github.com/lftroq/IT003.Q21.CTTN-W1-SortingReport.

IV. Trích dẫn

[1] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020)., "Array programming with NumPy," *Nature,* pp. 357-362, 2020.

[2] D. R. Musser, "Introspective Sorting and Selection Algorithms," *Software: Practice and Experience,* pp. 983-993, 1997.